EXERCISE 13.3

Hard: Implement a generic interpreter for `Free[F,A]`, given a `Monad[F]`. You can pattern your implementation after the `Async` interpreter given previously, including use of a tail-recursive `step` function.

```
def run[F[_],A](a: Free[F,A])(implicit F: Monad[F]): F[A]
```

What is the *meaning* of `Free[F,A]`? Essentially, it's a recursive structure that contains a value of type `A` wrapped in zero or more layers of `F`.[11] It's a monad because `flatMap` lets us take the `A` and from it generate *more* layers of `F`. Before getting at the result, an interpreter of the structure must be able to process all of those `F` layers. We can view the structure and its interpreter as coroutines that are interacting, and the type `F` *defines the protocol of this interaction*. By choosing our `F` carefully, we can precisely control what kinds of interactions are allowed.

13.4.2 A monad that supports only console I/O

`Function0` is not just the simplest possible choice for the type parameter `F`, but also one of the least restrictive in terms of what's allowed. This lack of restriction gives us no ability to reason about what a value of type `Function0[A]` might do. A more restrictive choice for `F` in `Free[F,A]` might be an algebraic data type that only models interaction with the console.

Listing 13.6 Creating our `Console` type

```
sealed trait Console[A] {
  def toPar: Par[A]
  def toThunk: () => A
}
case object ReadLine extends Console[Option[String]] {
  def toPar = Par.lazyUnit(run)
  def toThunk = () => run

  def run: Option[String] =
    try Some(readLine())
    catch { case e: Exception => None }
}
case class PrintLine(line: String) extends Console[Unit] {
  def toPar = Par.lazyUnit(println(line))
  def toThunk = () => println(line)
}
```

Interpret this `Console[A]` as a `Par[A]`.

Interpret this `Console[A]` as a `Function0[A]`.

Helper function used by both interpreters of `ReadLine`.

[11] Put another way, it's a tree with data of type `A` at the leaves, where the branching is described by `F`. Put yet another way, it's an abstract syntax tree for a program in a language whose instructions are given by `F`, with free variables in `A`.

A `Console[A]` represents a computation that yields an `A`, but it's restricted to one of two possible forms: `ReadLine` (having type `Console[Option[String]]`) or `PrintLine`. We bake two interpreters into `Console`, one that converts to a `Par`, and another that converts to a `Function0`. The implementations of these interpreters are straightforward.

We can now embed this data type into `Free` to obtain a restricted `IO` type allowing for only console I/O. We just use the `Suspend` constructor of `Free`:

```
object Console {
  type ConsoleIO[A] = Free[Console, A]

  def readLn: ConsoleIO[Option[String]] =
    Suspend(ReadLine)

  def printLn(line: String): ConsoleIO[Unit] =
    Suspend(PrintLine(line))
}
```

Using the `Free[Console,A]` type, or equivalently `ConsoleIO[A]`, we can write programs that interact with the console, and we reasonably expect that they don't perform other kinds of I/O:[12]

```
val f1: Free[Console, Option[String]] = for {
  _  <- printLn("I can only interact with the console.")
  ln <- readLn
} yield ln
```

Note that these aren't Scala's standard `readLine` and `println`, but the monadic methods we defined earlier.

This sounds good, but how do we actually run a `ConsoleIO`? Recall our signature for `run`:

```
def run[F[_],A](a: Free[F,A])(implicit F: Monad[F]): F[A]
```

In order to run a `Free[Console,A]`, we seem to need a `Monad[Console]`, which we don't have. Note that it's not possible to implement `flatMap` for `Console`:

```
sealed trait Console[A] {
  def flatMap[B](f: A => Console[B]): Console[B] = this match {
    case ReadLine => ???
    case PrintLine(s) => ???
  }
}
```

We must *translate* our `Console` type, which doesn't form a monad, to some other type (like `Function0` or `Par`) that does. We'll make use of the following type to do this translation:

Translate between any '`F[A]`' to '`G[A]`'.

This gives us infix syntax `F ~> G` for `Translate[F,G]`.

```
trait Translate[F[_], G[_]] { def apply[A](f: F[A]): G[A] }
type ~>[F[_], G[_]] = Translate[F,G]
```

[12] Of course, a Scala program could always technically have side effects but we're assuming that the programmer has adopted the discipline of programming without side effects, since Scala can't guarantee this for us.

```
val consoleToFunction0 =
  new (Console ~> Function0) { def apply[A](a: Console[A]) = a.toThunk }
val consoleToPar =
  new (Console ~> Par) { def apply[A](a: Console[A]) = a.toPar }
```

Using this type, we can generalize our earlier implementation of `run` slightly:

```
def runFree[F[_],G[_],A](free: Free[F,A])(t: F ~> G)(
                         implicit G: Monad[G]): G[A] =
  step(free) match {
    case Return(a) => G.unit(a)
    case Suspend(r) => t(r)
    case FlatMap(Suspend(r),f) => G.flatMap(t(r))(a => runFree(f(a))(t))
    case _ => sys.error("Impossible; `step` eliminates these cases")
  }
```

We accept a value of type `F ~> G` and perform the translation as we interpret the `Free[F,A]` program. Now we can implement the convenience functions `runConsoleFunction0` and `runConsolePar` to convert a `Free[Console,A]` to either `Function0[A]` or `Par[A]`:

```
def runConsoleFunction0[A](a: Free[Console,A]): () => A =
  runFree[Console,Function0,A](a)(consoleToFunction0)

def runConsolePar[A](a: Free[Console,A]): Par[A] =
  runFree[Console,Par,A](a)(consoleToPar)
```

This relies on having `Monad[Function0]` and `Monad[Par]` instances:

```
implicit val function0Monad = new Monad[Function0] {
  def unit[A](a: => A) = () => a
  def flatMap[A,B](a: Function0[A])(f: A => Function0[B]) =
    () => f(a())()
}
implicit val parMonad = new Monad[Par] {
  def unit[A](a: => A) = Par.unit(a)
  def flatMap[A,B](a: Par[A])(f: A => Par[B]) =
    Par.fork { Par.flatMap(a)(f) }
}
```

EXERCISE 13.4

Hard: It turns out that `runConsoleFunction0` isn't stack-safe, since `flatMap` isn't stack-safe for `Function0` (it has the same problem as our original, naive `IO` type in which `run` called itself in the implementation of `flatMap`). Implement `translate` using `runFree`, and then use it to implement `runConsole` in a stack-safe way.

```
def translate[F[_],G[_],A](f: Free[F,A])(fg: F ~> G): Free[G,A]
def runConsole[A](a: Free[Console,A]): A
```